Aoun Backend

Backend service for **Aoun**, a charitable giving platform that connects donors and recipients through a direct and organized donation workflow.

Overview

The backend is implemented with Node.js and Express. It uses MongoDB through Mongoose and exposes the application API through organized routes, controllers, services, repositories, models, DTOs, and middleware layers.

The project also contains Socket.io integration for real-time application events and communication.

Main capabilities

- Authentication and authorization.
- Phone-based authentication support.
- User and verification-related operations.
- Donation requests and item management.
- Safe hub management.
- Conversations and real-time communication.
- Notifications.
- Ratings and leaderboard functionality.
- Reports and moderation-related operations.
- Admin operations and dynamic settings.

Project structure

```
.
├── app.js              # Express application and middleware configuration
├── server.js           # Server startup and Socket.io integration
├── config/             # Application configuration
├── controllers/        # Request handlers
├── dtos/               # Data-transfer objects and contracts
├── integrations/       # External service integrations
├── jobs/               # Background jobs
├── middlewares/        # Authentication, validation, and shared middleware
├── models/             # Mongoose models
├── repositories/       # Data-access layer
├── routes/             # API route definitions
├── scripts/            # Utility and seed scripts
├── services/           # Business logic
```

```
├── socket/             # Socket.io configuration and events
└── utils/              # Shared utilities
```

Available route groups

The current route layer includes modules for authentication, administration, conversations, donation requests, hubs, items, leaderboard, notifications, phone authentication, ratings, reports, and settings.

Requirements

- Node.js.
- MongoDB.
- The environment variables required by the configuration and integration files.
- Credentials for any external services enabled by the deployment configuration.

Installation

```
npm install
```

Create an environment file according to the variables read by the project configuration, then start the server with:

```
node server.js
```

For development, use the project's preferred Node.js watch or process-management command.

Security and infrastructure

The application includes security-oriented middleware and protections such as Helmet, CORS configuration, cookie parsing, rate limiting, request handling controls, and Socket.io server integration. Production deployments should use secure secrets, HTTPS, restricted CORS origins, and environment-specific credentials.

Important notes

- Do not commit `.env` files, private keys, Firebase credentials, JWT secrets, or third-party API credentials.
- Keep database access and business rules inside the appropriate repository and service layers.
- Validate and authorize every state-changing request on the server.
- Real-time events should be treated as untrusted input and must be protected by server-side authorization checks.

Available package script

The repository currently contains the default test placeholder script in `package.json`. The server is started directly through `server.js`.